

Notes — Go

Langages compilés vs interprétés

Langages compilés : - Performants (plus près de la machine) - Rigides - Risques de fuites mémoire
- Ce n'est pas pour les amateurs

Langages interprétés : - Souples - Performants - Polyvalents

Origine de Go

Go a été créé par d'anciens développeurs C++ qui ont adopté une approche disruptive pour apporter un vrai changement.

- **Go** → composition
- **PHP** → héritage

Avantages de Go

- Simplicité de développement
- Rapidité d'exécution et de compilation
- Beaucoup de contraintes qui facilitent le développement
- Pas de try/catch : tout est géré par des contraintes
- **Goroutines** → multi-threading amélioré (plus de deadlocks)

Globalement, Go est pensé pour être pragmatique et efficace, ce qui apporte une très grande stabilité et compatibilité. C'est un excellent compromis entre simplicité, performance, et rapidité de développement et de compilation.

Cas d'usage

- **Go** est idéal pour : microservices, API REST, applications CLI, IaaS
- **Rust** est idéal pour : applications nécessitant beaucoup de sécurité, drivers, et hautes performances

Point d'entrée

Le package main est le point d'entrée de toute application en Go.

voilà le petit code fait dans le cours :

```
package main

import "fmt"
import "math"
import "time"

func main() {
    // affiche "Hello, World!" dans la console
    fmt.Println("Hello, World!")
}
```

```
// affiche le résultat de 64 élevé à la puissance 8
fmt.Println(math.Pow(64, 8))

// affiche la date et l'heure actuelles
fmt.Println(time.Now().Format("Today is Monday, January 2, 2006 and it's 3:04 PM"))

// age calculé à partir de la date de naissance
birthDate := time.Date(2003, time.May, 20, 0, 0, 0, 0, time.UTC)
age := time.Now().Year() - birthDate.Year()
fmt.Printf("Age calculated from birth date: %d years", age)
}
```

Quel est la différence entre `go run` et `go build`? `go run` compile et exécute le code immédiatement, tandis que `go build` compile le code et génère un exécutable sans l'exécuter.

La commande `go build` se construit de cette manière : `go build [flags] [packages]` les paramètres les plus courants sont : - `-o` : spécifie le nom de l'exécutable généré - `-v` : affiche les packages compilés - `-race` : active la détection des conditions de course - `-ldflags` : permet de passer des options au linker, comme l'injection de variables au moment de la compilation. par exemple : `go build -ldflags "-s -w"` pour réduire la taille de l'exécutable en supprimant les symboles de débogage. - `-tags` : permet de spécifier des tags de build pour inclure ou exclure du code conditionnellement.

Il est également possible d'ajouter des variables d'environnement pour personnaliser le comportement de la compilation, comme `GOOS` et `GOARCH` pour cibler des systèmes d'exploitation et des architectures spécifiques.

Les types en Go

Entiers : `int`, `int8`, `int16`, `int32`, `int64` (signés) et `uint`, `uint8`, `uint16`, `uint32`, `uint64` (non signés) - `int` et `uint` sont des types d'entiers dont la taille dépend de l'architecture de la machine (32 ou 64 bits).

Flottants : `float32`, `float64`

Booleans : `bool`

Strings : `string` on peut concaténer et additionner des strings. Ils sont également indexables et immuables.

Spéciaux : `byte` (alias pour `uint8`), `rune` (alias pour `int32`, représente un point de code Unicode), `Nil` (valeur zéro pour les pointeurs, interfaces, slices, channels, maps et fonctions)

Les variables en Go

- `var` : déclaration de variable avec un type explicite
- `:=` : déclaration et initialisation de variable avec inférence de type
- `const` : déclaration de constante

- `iota` : générateur de constantes incrémentielles. `iota` commence à 0 dans chaque bloc `const` et s'incrémente de 1 à chaque ligne :

```
const (  
  A = iota // 0  
  B = iota // 1  
  C = iota // 2  
)
```

Il y a aussi une version plus concise :

```
const (  
  A = iota // 0  
  B        // 1 (iota est implicite)  
  C        // 2 (iota est implicite)  
)
```

Voici un cas d'utilisation classique :

```
type Jour int  
  
const (  
  Lundi Jour = iota // 0  
  Mardi      // 1  
  Mercredi   // 2  
  Jeudi      // 3  
  Vendredi   // 4  
  Samedi     // 5  
  Dimanche   // 6  
)
```